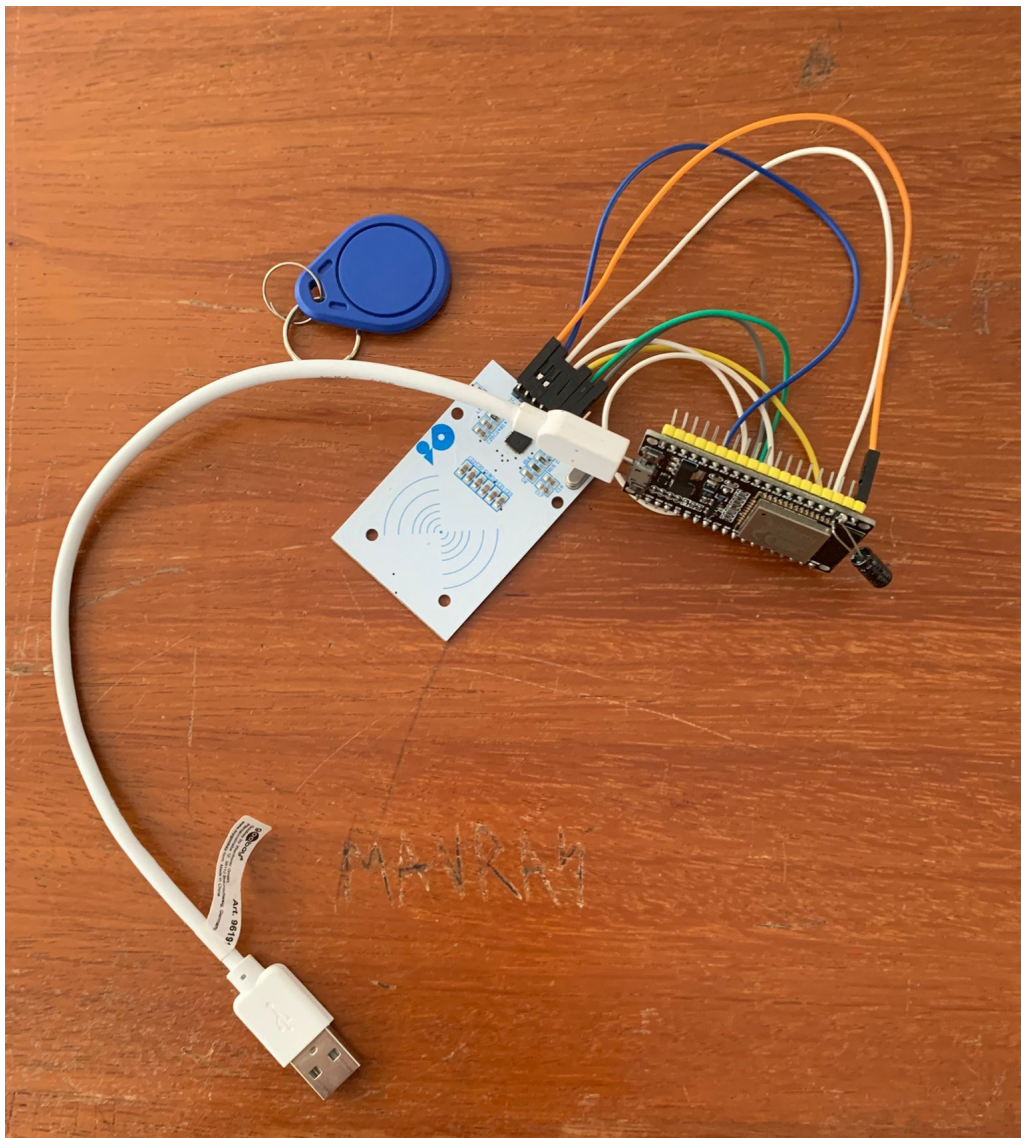


Projeto Automação II

Licenciatura em Automação Industrial



Tomás Espincho (110486)

19 de junho de 2026

Índice

1	Introdução	2
2	Explicação do Código	2
2.1	Código do ESP32	2
2.1.1	Conversão do UID para Hexadecimal	2
2.1.2	Construção da Mensagem e Publicação MQTT	2
2.2	Código de Testes (python e logica NodeRed)	3
2.2.1	Simulação e Validação em Python	3
2.2.2	Lógica em Node-RED	3
3	Resultados e Conclusão	4
	Referências	5

1 Introdução

Neste projeto, o objetivo consiste em desenvolver um programa que lê uma tag RFID e a publica num broker MQTT. A arquitetura principal baseia-se num leitor RFID (MFRC522) ligado a um ESP32, que comunica com o servidor MQTT por Wi-Fi e, no Docker para alojar os serviços do broker e da dashboard.

O processo começa por ler uma tag RFID. O ESP32 usa a biblioteca MFRC522.h para extrair o UID e a biblioteca NTPClient para registar a hora exata da leitura. Depois, esta informação é estruturada num JSON e enviada para o broker MQTT. Por fim, o broker envia a informação para todos os subscritores do tópico, seja o script Python ou a dashboard.

2 Explicação do Código

O código desenvolvido neste trabalho é apresentado e explicado nas subsecções seguintes, divididas por linguagem de programação.

2.1 Código do ESP32

O código do ESP32 é responsável pela interação com o leitor RFID, processamento dos dados e comunicação MQTT. O código completo encontra-se disponível no repositório GitHub e(ou) no zip do projeto. Nesta secção, estão apenas explicitas as partes mais estruturais do código.

2.1.1 Conversão do UID para Hexadecimal

Após a leitura de uma tag, o UID é obtido como um *array* de *bytes*. Para garantir a correta formatação dos dados no sistema, este valor é convertido numa *string* em formato hexadecimal.

A lógica do módulo baseia-se na documentação e exemplos práticos do sensor RC522 [1], [2]

```
1 String tagUID = "";
2 for (byte i = 0; i < mfrc522.uid.size; i++) {
3   tagUID.concat(String(mfrc522.uid.uidByte[i] < 0x10 ? " 0" : " "))
4   ;
5   tagUID.concat(String(mfrc522.uid.uidByte[i], HEX));
6 }
7 tagUID.toUpperCase();
8 String finalUID = tagUID.substring(1);
```

Listing 1: Conversão do UID para formato hexadecimal

2.1.2 Construção da Mensagem e Publicação MQTT

Com o UID processado, o microcontrolador obtém o *timestamp* em formato UNIX através do cliente NTP [3], [4]. Os dados obtidos (identificação do dispositivo, UID da *tag* e *timestamp*) são aglomerados para construir uma mensagem (*payload*) em formato JSON. Por fim, esta mensagem estruturada é publicada no *broker* MQTT.

```

1 timeStamp = timeClient.getEpochTime();
2
3 String payload = "{\"dispositivo\" : \"Esp32\", \"uid\": \"\" +
    finalUID + "\", \"timestamp\": \"\" + String(timeStamp) + \"\"}";
4
5 client.publish("aii/projeto/rfid", payload.c_str());

```

Listing 2: Geração da mensagem JSON e envio via MQTT

2.2 Código de Testes (python e logica NodeRed)

Para validar o fluxo de dados sem dependência do *hardware* e garantir a integração com a *dashboard*, foram desenvolvidos *scripts* de teste em Python e um fluxo de dados através de Node-RED.

2.2.1 Simulação e Validação em Python

O processo de simulação baseia-se em dois *scripts* que recorrem à biblioteca *paho-mqtt*, que já tinha utilizado para o projeto final de Licenciatura.

O *script sender.py* foi desenvolvido para funcionar no lugar do ESP32, enviando assim um *payload* JSON para o *broker* MQTT no tópico *aii/projeto/rfid*.

```

1 dados_sensor = {
2     "dispositivo": "Esp32",
3     "uid": "A1B2C3D4",
4     "status": "lido",
5     "timestamp": int(time.time()),
6 }
7 mensagem_json = json.dumps(dados_sensor)
8 client.publish(topico, mensagem_json)

```

Listing 3: Geração e envio de dados simulados

O *script reciver.py* subscrive ao tópico do projeto e apresenta todas as mensagens no topico *aii/projeto/rfid*. Ao identificar uma nova publicação, dá print ao conteúdo com a chave "uid".

2.2.2 Lógica em Node-RED

A passagem de dados entre o MQTT e o site é feita pelo Node-RED, que está alojado num contentor Docker na porta 1880. As Nodes desenvolvidas foram mapeadas através do diagrama da Figura 1.

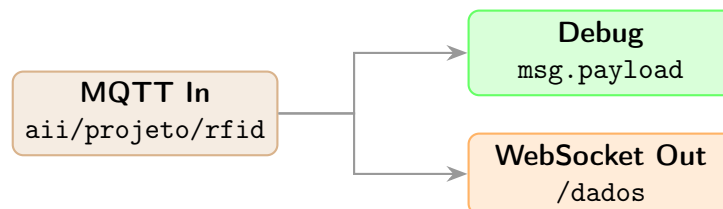


Figura 1: Fluxo de encaminhamento de dados implementado no Node-RED.

A organização é feita da seguinte maneira:

- **Entrada MQTT:** O nó *MQTT in* subscrive o tópico *aii/projeto/rfid* no broker.

- **Nó de Monitorização (Debug):** imprime o *payload* recebido para a consola de debug do Node-RED, para validar a chegada das mensagens.
- **Saída WebSocket:** Envia a mensagem integral para o nó *WebSocket out*, que disponibiliza os dados na rota */dados*, que permite receber os dados instantaneamente no *site*.

3 Resultados e Conclusão

Os objetivos propostos para este projeto foram totalmente atingidos. Foi validado todo o fluxo de dados, desde a leitura física das *tags* RFID pelo ESP32 até ao processamento do UID em formato hexadecimal e sincronização temporal via cliente NTP [3], [4].

Todos os testes complementares em ambiente Docker/Python foram verificados. O *broker* Mosquitto assegurou a receção e distribuição contínua dos pacotes JSON sem ocorrência de perdas, mesmo este estando configurado para usar QoS 0. O fluxo no Node-RED funcionou, convertendo as publicações MQTT em *WebSockets* para poderem ser apresentados instantaneamente os dados na *dashboard*.

Em suma, o projeto permitiu consolidar com sucesso os conceitos práticos dados nas aulas de Automação II.

Referências

- [1] *Interfacing RC522 RFID Module With Arduino : 5 Steps - Instructables*. URL: <https://www.instructables.com/Interfacing-RC522-RFID-Module-With-Arduino/>
- [2] *Arduino RFID Module RC522 - How to Use - YouTube*. URL: <https://www.youtube.com/watch?v=cFK87MJ96A8>
- [3] *Getting Date and Time with ESP32 (NTP Client) | Random Nerd Tutorials*, Acedido em: 19 de junho de 2026. URL: <https://randomnerdtutorials.com/esp32-ntp-client-date-time-arduino-ide/>
- [4] *PaulStoffregen/Time: Time library for Arduino*, Repositório GitHub. URL: <https://github.com/PaulStoffregen/Time/tree/master>